

Sort a Linked List

My Personal DSA Notes — Merge Sort on a Singly Linked List

Goal: Sort the nodes of a linked list and return the head of the updated sorted list.

1. Problem Statement

You are given the head of a linked list. Every node contains an integer. Sort all the integers and return the head of the updated linked list.

Example input



Sorted result



2. Intuition

The straightforward idea is to take every value from the linked list, store it in an array/list, sort that array, and write the sorted values back into the existing nodes. The optimized idea is to use **merge sort directly on the linked list**. Linked lists are well suited to merging because nodes can be rearranged by changing next pointers.

3. Brute Force — Array/List

1) Traverse the list and store every value. 2) Sort the array/list. 3) Traverse again and replace node values. 4) Return the same head.

```
List<Integer> arr = new ArrayList<>();
ListNode temp = head;

while (temp != null) {
    arr.add(temp.data);
    temp = temp.next;
}

Collections.sort(arr);

int i = 0;
temp = head;

while (temp != null) {
    temp.data = arr.get(i);
    i++;
    temp = temp.next;
}

return head;
```

Complexity from the video: $O(n) + O(n \log n) + O(n) = O(n \log n)$ time. Extra array/list space = $O(n)$.

4. Why Optimize?

The interviewer may reject the $O(n)$ external space. The video therefore moves the sorting operation inside the linked list itself and chooses merge sort. The video notes that merge sort is easier to implement on a linked list than quick sort in this setting.

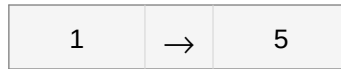
5. Merge Sort on a Linked List

Merge sort has three main stages: **find the middle** → **split** → **recursively sort and merge**.

Left portion



Right portion



6. Important Even-Length Edge Case

The video points out that the usual slow/fast middle routine returns the **second middle** for an even-length list. For this merge-sort split, we need the **first middle**. The required modification is to start fast at **head.next** while slow starts at head.

This lets slow stop at the first middle, after which middle.next can be set to null to separate the two lists.

7. Finding the First Middle and Splitting

```
ListNode slow = head;
ListNode fast = head.next;

while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
}

ListNode rightHead = slow.next;
slow.next = null; // Separate left and right lists
```

The left list starts at head. The right list starts at middle.next. Setting middle.next = null is essential because the recursive calls must receive two independent lists.

8. Recursive Structure

An empty list or a one-node list is already sorted, so it is the base case.

```
if (head == null || head.next == null) {
    return head;
}
```

Otherwise: find the first middle, split the list, recursively sort both halves, and merge the two sorted halves.

9. Complete Java Solution

```
class Solution {

    private ListNode findMiddle(ListNode head) {
        ListNode slow = head;
        ListNode fast = head.next;

        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }

        return slow;
    }

}
```

```

private ListNode mergeTwoSortedLists(ListNode list1, ListNode list2) {
    ListNode dummy = new ListNode(-1);
    ListNode temp = dummy;

    while (list1 != null && list2 != null) {
        if (list1.data <= list2.data) {
            temp.next = list1;
            list1 = list1.next;
        } else {
            temp.next = list2;
            list2 = list2.next;
        }
        temp = temp.next;
    }

    if (list1 != null) {
        temp.next = list1;
    } else {
        temp.next = list2;
    }

    return dummy.next;
}

public ListNode sortList(ListNode head) {

    // Empty or one-node list is already sorted.
    if (head == null || head.next == null) {
        return head;
    }

    // Find the first middle for the correct split.
    ListNode middle = findMiddle(head);

    // Save the right half before breaking the link.
    ListNode rightHead = middle.next;

    // Separate the left and right lists.
    middle.next = null;

    // Recursively sort both halves.
    ListNode leftSorted = sortList(head);
    ListNode rightSorted = sortList(rightHead);

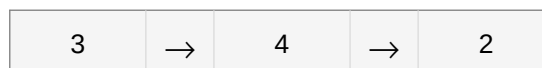
    // Merge the two sorted halves.
    return mergeTwoSortedLists(leftSorted, rightSorted);
}

```

10. Pointer Picture of the Split

For $3 \rightarrow 4 \rightarrow 2 \rightarrow 1 \rightarrow 5$, the first split can be:

Left half



Right half



Each half is then split again until every sub-list contains one node. Those one-node lists are the base cases.

11. Recursive Dry Run

Input: $3 \rightarrow 4 \rightarrow 2 \rightarrow 1 \rightarrow 5$

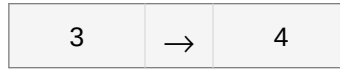
First split: [3,4,2] and [1,5].

Left side: [3,4,2] $\rightarrow$ [3,4] + [2] $\rightarrow$ [3] + [4] $\rightarrow$ merge to [3,4] $\rightarrow$ merge with [2] to get [2,3,4].

Right side: [1,5] $\rightarrow$ [1] + [5] $\rightarrow$ merge to [1,5].

Final merge: [2,3,4] + [1,5] $\rightarrow$ [1,2,3,4,5].

Merge result



Left sorted result



Right sorted result



Final result



12. Why the Merge Works

When recursion returns, both halves are sorted. The merge compares the current node of each list, attaches the smaller node, advances that list, and continues. Existing nodes are reused; no array is required.

The dummy node is only a temporary starting point. The actual merged list begins at **dummy.next**.

13. Why Return the Head?

In linked-list merge sort, the head can change after a merge. Therefore the recursive function must return the head of the updated sorted list. The video explicitly highlights this distinction.

return head means returning the node reference that represents the whole chain starting from that node.

14. Time Complexity

The video explains that merge sort has logarithmic recursion depth because the list is repeatedly divided into halves. At each level, merging is linear, and finding the middle also takes linear time for the current portion.

The video therefore gives the overall time as approximately **$O(n \log n)$** . The middle-finding work adds a linear scan at each recursion level, but the asymptotic result remains $O(n \log n)$.

15. Space Complexity

The video states **$O(1)$ extra space** for the linked-list approach because the merge is performed in place and no external array is used.

Recursion-stack note: this implementation is recursive, so the call stack has $O(\log n)$ depth. The video presents $O(1)$ as the external/auxiliary data-structure space for the in-place linked-list operations.

16. Brute Force vs Optimized

Approach	Time	External Space	Idea
Array/List	$O(n \log n)$	$O(n)$	Copy → sort → write back
Merge Sort	$O(n \log n)$	$O(1)^*$	Split → sort → merge

*The video reports $O(1)$ external space for the in-place linked-list method; the recursive call stack itself is $O(\log n)$.

17. Edge Cases

- Empty list → return null.
- One node → already sorted.
- Two nodes → split into two single-node lists and merge.
- Even length → use `fast = head.next` so the first middle is selected.
- Duplicate values → merge handles them normally.
- Negative values → comparisons work normally.

18. Key Lines to Remember

```
// First-middle trick
ListNode slow = head;
ListNode fast = head.next;

// Split
ListNode rightHead = middle.next;
middle.next = null;

// Recursive sorting
ListNode leftSorted = sortList(head);
ListNode rightSorted = sortList(rightHead);

// Combine
return mergeTwoSortedList(leftSorted, rightSorted);
```

19. Quick Revision

Brute: array/list + sort + rewrite → $O(n \log n)$, $O(n)$ extra space.

Optimized: merge sort on the linked list.

Split: slow/fast with `fast = head.next` to get the first middle for even length.

Break: `middle.next = null`.

Base case: empty or one node.

Combine: merge two sorted linked lists in place.

Time: $O(n \log n)$.

External space: $O(1)$ for in-place operations; recursive stack $O(\log n)$.

20. Final Intuition

Break → sort → merge. Keep splitting the linked list into smaller lists until every list has one node. Then merge those sorted lists back together. The most important linked-list details are selecting the first middle for an even-length list, disconnecting the halves with `middle.next = null`, and reusing nodes during merge.